

Informatics Engineering Faculty

Academic Year: 2021- 2022

5th Year Students

Course: Project Management



Lecture -4-

Agile Principles, Jira Searching and JQL

Prepared by: Dr. Abdo Darbooli

Acknowledgment

- This lecture is in based on:
 - *Agile with Atlassian Jira, offered by Atlassian University*

Outline

- Agile Manifesto
 - *Value Statements*
 - *Agile Principles*
- Jira Searching
 - *Quick Search*
 - *Basic Search*
- JQL
 - *Writing JQL*
 - *Ordering Results*
 - *Autocomplete*
 - *Functions*
 - *Time Unit Qualifiers*
 - *Operators*

Agile Manifesto

Manifesto for Agile Development

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Kent Beck	James Grenning	Robert C. Martin
Mike Beedle	Jim Highsmith	Steve Mellor
Arie van Bennekum	Andrew Hunt	Ken Schwaber
Alistair Cockburn	Ron Jeffries	Jeff Sutherland
Ward Cunningham	Jon Kern	Dave Thomas
Martin Fowler	Brian Marick	

© 2001, the above authors
this declaration may be freely copied in any form,
but only in its entirety through this notice.

Agile Manifesto Value Statements

- The following are some takeaway principles

1. Empower the team
 - **"Individuals and interactions** over processes and tools"
2. Embrace change
 - **"Responding to change** over following a plan"
3. Partner with the customer
 - **"Customer collaboration** over contract negotiation"
4. Plan, develop and deliver incrementally
 - **"Working software** over comprehensive documentation"

Agile Principles Behind Manifesto (1/12)

"Our highest priority is to satisfy the customer through early and continuous delivery of valuable software."

In other words:

- Plan, develop and deliver incrementally
- Obtain fast feedback
- Focus on value

Agile Principles Behind Manifesto (2/12)

"Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage."

In other words:

- Embrace change
 - It provides a competitive advantage

Agile Principles Behind Manifesto (3/12)

"Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale."

In other words:

- Plan, develop and deliver incrementally
 - With a preference to a higher frequency
- Obtain fast feedback

Agile Principles Behind Manifesto (4/12)

"Business people and developers must work together daily throughout the project."

In other words:

- Collaborate to create shared understanding

Agile Principles Behind Manifesto (5/12)

"Build projects around motivated individuals.
Give them the environment and support they need,
and trust them to get the job done."

In other words:

- Select motivated individuals
- Empower the team

Agile Principles Behind Manifesto (6/12)

"The most efficient and effective method of conveying information to and within a development team is face-to-face conversation."

In other words:

- Prefer conversations for conveying information

Agile Principles Behind Manifesto (7/12)

"Working software is the primary measure of progress."

In other words:

- Completed work items are the primary measure of progress

Agile Principles Behind Manifesto (8/12)

"Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely."

In other words:

- Maintain a sustainable pace
 - It is a byproduct of the proper use of agile processes

Agile Principles Behind Manifesto (9/12)

"Continuous attention to technical excellence and good design enhances agility."

In other words:

- Don't compromise on quality
- Continuously refactor to maintain agility

Agile Principles Behind Manifesto (10/12)

"Simplicity--the art of maximizing the amount of work not done--is essential."

In other words:

- Continuously strive for simplicity
- Focus on value, eliminate waste

"It takes a lot of hard work to make something simple..."

Steve Jobs

"I would have written a shorter letter, but I did not have the time."

Blaise Pascal

"Simplicity is the ultimate sophistication."

Leonardo da Vinci

Agile Principles Behind Manifesto (11/12)

"The best architectures, requirements, and designs emerge from self-organizing teams."

In other words:

- Teams should self-organize
 - It results in the best outcome

Agile Principles Behind Manifesto (12/12)

"At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly."

In other words:

- Continuously inspect and adapt

Principles and Ideas from Agile Manifesto

1. Empower the team
 - Select motivated individuals
 - Teams should self-organize
 - Collaborate to create shared understanding
2. Embrace change
 - Partner with the customer
 - Obtain fast feedback
 - Continuously inspect and adapt
3. Plan, develop and deliver incrementally
 - Prefer conversations for conveying information
 - Continuously refactor to maintain agility
 - Maintain a sustainable pace
 - Completed work items are the primary measure of progress
4. Focus on value
 - Eliminate waste
 - Continuously strive for simplicity
 - Don't compromise on quality

Jira Searching

Searching Overview

- We can view a project's progress using boards, search, reports, dashboards



Boards



Search



Reports



Dashboards

Searching

- **Quick search** is a text search for issues, boards and projects
- **Basic search** uses a user interface elements to search for issues
- **Advanced search** uses a text-based statements to search for issues
- **Filters** are saved searches
- **Quick filters** limit the issues visible on the board

Quick Search with Keywords

- Use NOT keyword to exclude items

The image displays three side-by-side screenshots of a search interface, each showing a list of issues. Each panel has a search bar at the top with a magnifying glass icon and a search term.

- Panel 1: Search for 'item'**
 - Search bar: Q item
 - ISSUES 3
 - Results:
 - PROJ-3 add item 3
projectA • Recently viewed
 - PROJ-2 add item 2
projectA • Recently viewed
 - PROJ-1 add item 1
projectA • Recently viewed
- Panel 2: Search for 'item NOT 1'**
 - Search bar: Q item NOT 1
 - ISSUES 2
 - Results:
 - PROJ-3 add item 3
projectA • Updated 2 minutes ago
 - PROJ-2 add item 2
projectA • Updated 2 minutes ago
- Panel 3: Search for 'item OR sample'**
 - Search bar: Q item OR sample
 - ISSUES 4
 - Results:
 - PROJ-3 add item 3
projectA • Updated 3 minutes ago
 - PROJ-4 create sample data
projectA • Updated 2 minutes ago
 - PROJ-2 add item 2
projectA • Updated 3 minutes ago
 - PROJ-1 add item 1
projectA • Updated 3 minutes ago

Basic Search

- User interface

Project: All ▼ Type: All ▼ Status: All ▼ Assignee: All ▼ [+ More](#) [Search](#) [Switch to JQL](#)

JQL

Basic and Advanced JQL Search

basic

Search Save as    Share

projectA ▼ Type: All ▼ Status: All ▼ Assignee: All ▼ + More Contains text **Search** Switch to JQL

advanced/JQL

Search Save as    Share  Export

 project = PROJ order by lastViewed DESC  **Search** Switch to basic

JQL

- Jira Query Language (JQL)
- Uses a structured query language (SQL)-like syntax
- Searches issues only
 - *simple*



Basic vs Advanced/JQL Search

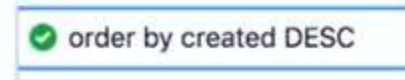
Basic search

- User-friendly interface
- Queries can be complex

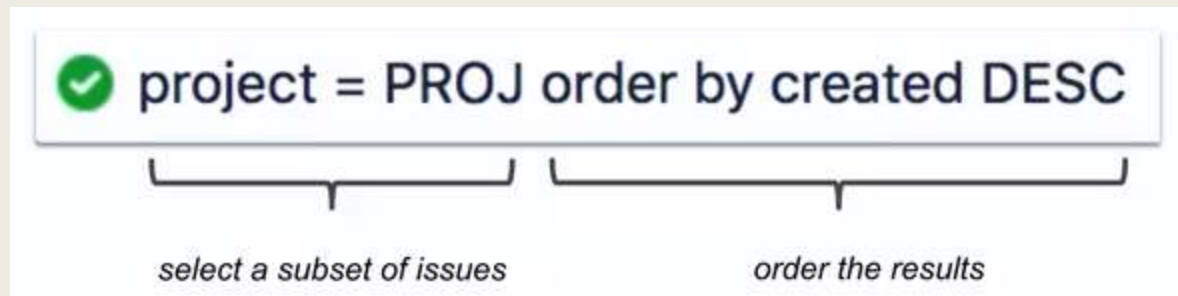


Advanced search

- Uses JQL
- Most powerful search method
- JQL can be used in automation scripts

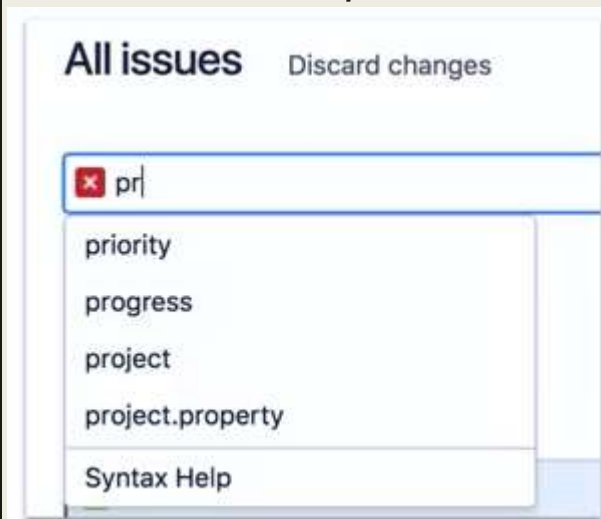


Writing JQL

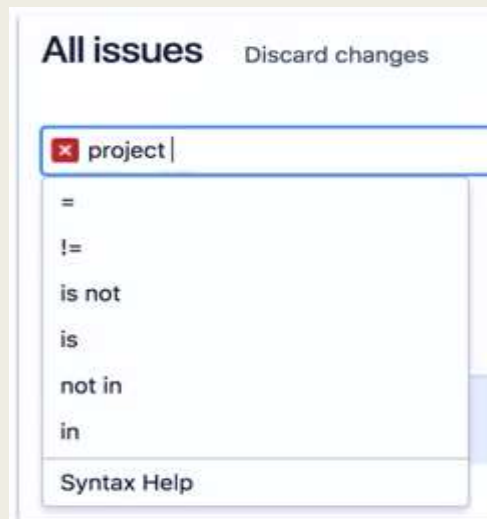


JQL with Autocomplete

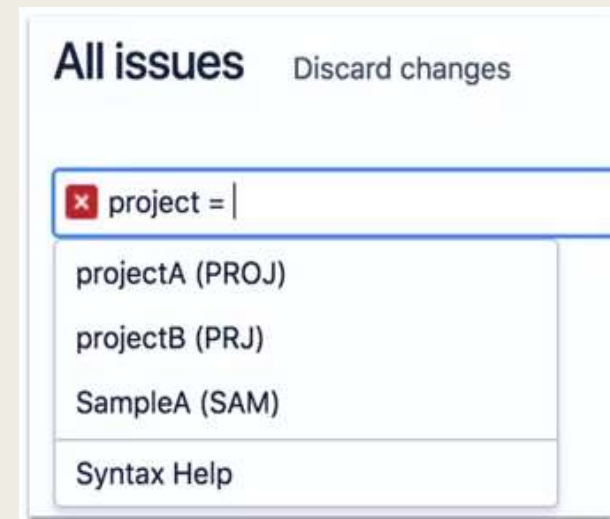
- Creating a JQL query from scratch
- Project is a fieldname in a Jira issue
- Autocomplete will show up to 15 matches
- Jira provides a list of choices of what should be next in the query
 - *operators*



This screenshot shows the Jira JQL interface with the text 'pr' entered in the search bar. The autocomplete dropdown lists the following suggestions: 'priority', 'progress', 'project', 'project.property', and 'Syntax Help'.



This screenshot shows the Jira JQL interface with the text 'project' entered in the search bar. The autocomplete dropdown lists the following suggestions: '=', '!=', 'is not', 'is', 'not in', 'in', and 'Syntax Help'.



This screenshot shows the Jira JQL interface with the text 'project =' entered in the search bar. The autocomplete dropdown lists the following suggestions: 'projectA (PROJ)', 'projectB (PRJ)', 'SampleA (SAM)', and 'Syntax Help'.

Automatically Adding Order By

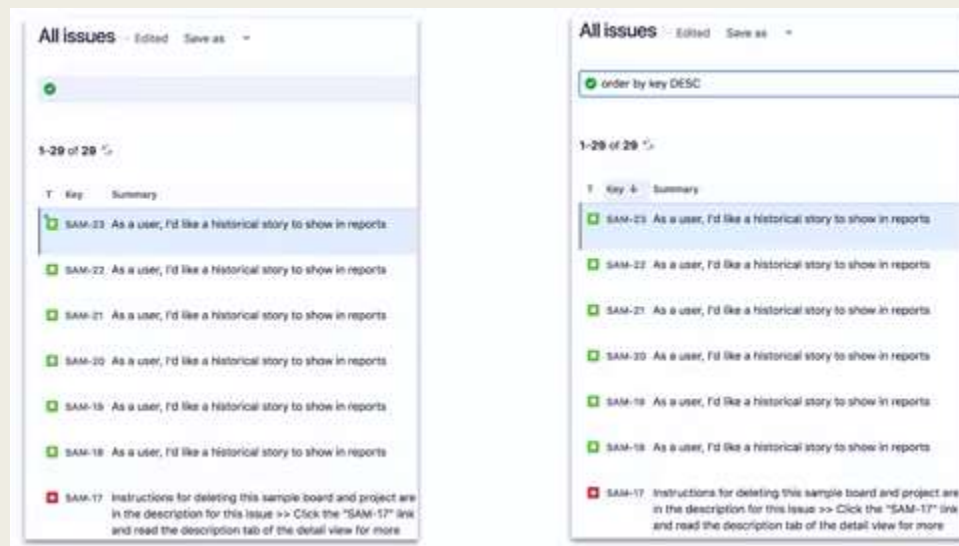


The screenshot shows a Jira search results page. At the top, there is a search bar with the query "project = projectA ORDER BY updated DESC". Below the search bar, the results are displayed in a table. The table has columns: T, Key, Summary, Assignee, Reporter, P, Status, Resolution, Created, Updated, and Due. A red arrow points to the "Updated" column header, which is currently sorted in descending order (indicated by a downward arrow). The first row of the table shows a task with key "PROJ-1" and summary "add item 1", assigned to Steve Byrnes, with a status of "BACKLOG" and a resolution of "Unresolved".

T	Key	Summary	Assignee	Reporter	P	Status	Resolution	Created	Updated	Due
	PROJ-1	add item 1	Steve Byrnes	Steve Byrnes		BACKLOG	Unresolved	20/Mar/20	05/Sep/20	

Order By

- The default order by clause is “order by key DESC”

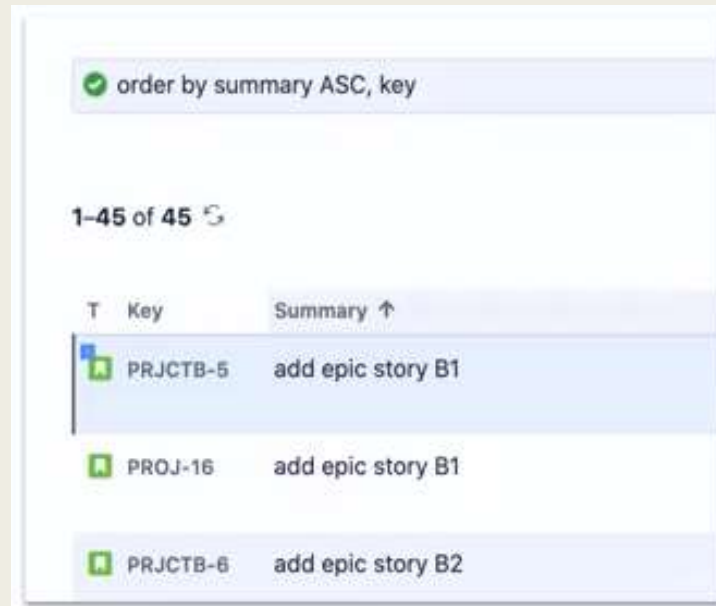


All Issues

The screenshot displays the Jira Software 'All Issues' view. The top navigation bar includes 'Jira Software', 'Your work', 'Projects', 'Filters', 'Dashboards', 'People', 'Apps', and a 'Create' button. A search bar is located on the right. The left sidebar lists various filters: 'Search issues', 'STATUS', 'My in progress', 'OTHER', 'My open issues', 'Reported by me', 'All issues' (highlighted), 'Open issues', and 'Done issues'. The main area shows 'All issues' with a 'Save as' button and a 'Search' button. Below the search bar, there is a table of issues. A red arrow points to the 'Switch to basic' button.

T	Key	Summary	Assignee	Reporter	P	Status	Resolution	Created	Updated
1	PRJCT2-6	add item 6	Unassigned	Steve Byrnes	↑	SELECTED FOR DEV...	Unresolved	13/Sep/20	13/Sep/20
2	PRJCT2-5	add item 5	Unassigned	Steve Byrnes	↑	SELECTED FOR DEV...	Unresolved	13/Sep/20	13/Sep/20
3	PRJCT2-4	add item 4	Unassigned	Steve Byrnes	↑	REVIEW	Unresolved	13/Sep/20	13/Sep/20
4	PRJCT3-1	add item 1	Unassigned	Steve	↑	BACKLOG	Unresolved	05/Sep/20	05/Sep/20

Ordering By Multiple Columns



A screenshot of a Jira issue list interface. At the top, a green checkmark icon is followed by the text "order by summary ASC, key". Below this, it says "1-45 of 45" with a refresh icon. The table has three columns: "T" (a small blue square icon), "Key", and "Summary" (with an upward arrow indicating ascending order). The table contains three rows of data, each with a green square icon in the "T" column.

T	Key	Summary ↑
	PRJCTB-5	add epic story B1
	PROJ-16	add epic story B1
	PRJCTB-6	add epic story B2

Issue Selection Clauses

- An issue selection clause is used to limit or filter the issues that are returned in the results
- Using functions -> no hard-coded information

```
<field name> <operator> <field value>  
project = projectA
```

```
<field name> <operator> <function>  
assignee = currentUser()
```

Time-Based Functions

- `startOfDay()`
- `startOfWeek()`
- `startOfMonth()`
- `startOfYear()`
- `endOfDay()`
- `endOfWeek()`
- `endOfMonth()`
- `endOfYear()`
- `now()`
- `currentLogin()`
- `lastLogin()`

Issues created since the start of today

```
created > startOfDay()
```

Time Unit Qualifier

- If you add a minus sign to the string, you are searching back in time.
- Using the plus sign or leaving off the sign altogether means that you are searching forward in time.

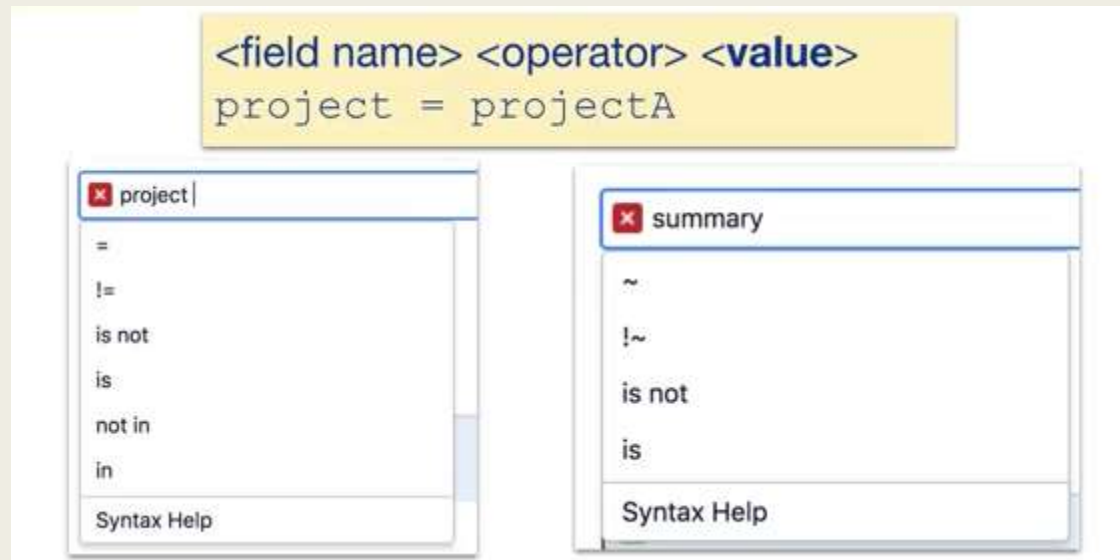
(+|-)nn(y|M|w|d|h|m)

created in the last 2 days (48 hours):
`created > -2d`

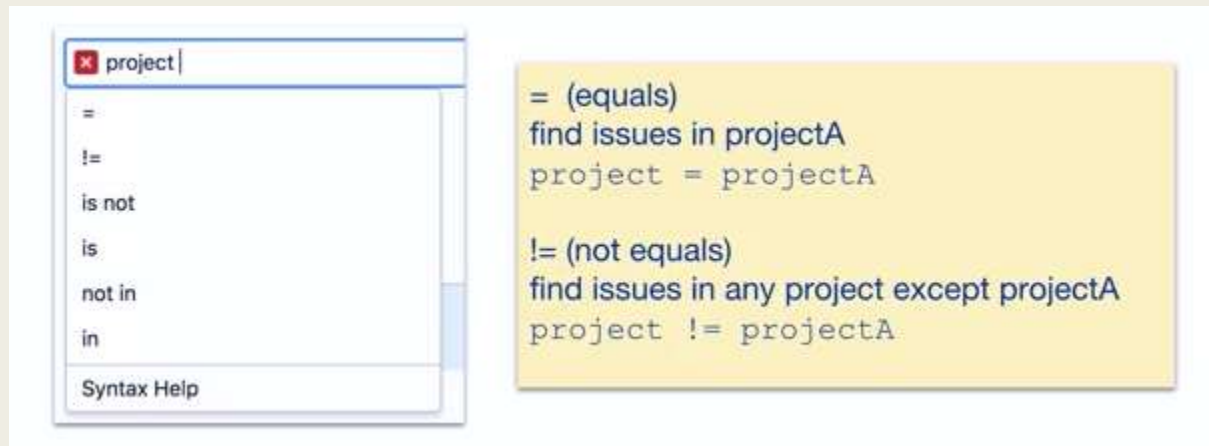
created since the start of day 2 days ago:
`created > startOfDay(-2d)`

created since the 15th of this month
`created > startOfMonth(+14d)`

Operator Autocomplete



Operators: equals AND not equals



The image shows a search interface with a dropdown menu for operators. The dropdown is titled 'project |' and lists the following operators: '=', '!=', 'is not', 'is', 'not in', and 'in'. Below the list is a 'Syntax Help' link. To the right of the dropdown, a yellow box contains examples for the '=' and '!=' operators.

= (equals)
find issues in projectA
`project = projectA`

!= (not equals)
find issues in any project except projectA
`project != projectA`

Operators: < <= > >=



The screenshot shows a search bar with the text "created" and a dropdown menu with the following suggestions: "=", "!=", "<=", ">=", ">", "<", "is not", "is", "not in", "in", and a "Syntax Help" link at the bottom.

> (greater than)
find issues created today
`created > startOfDay()`

>= (greater than or equals)
find issues with 3 or more story points
`"Story Points" >= 3`

< (less than)
find issues created more than a week ago
`created < -1w`

<= (less than or equals)
find issues with 3 or less story points
`"Story Points" <= 3`

Operators: in AND not in

- Used with a comma-separated list of values in the JQL



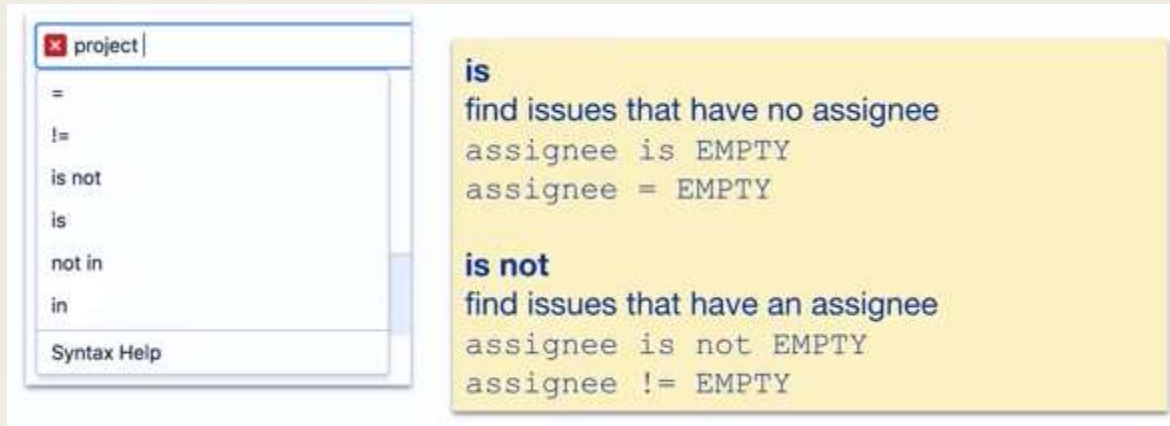
The image shows a JQL editor interface on the left and a yellow callout box on the right. The editor has a search bar with 'project' and a dropdown list of operators: '=', '!=', 'is not', 'is', 'not in', and 'in'. A 'Syntax Help' link is at the bottom of the dropdown. The yellow box contains two sections: 'in' with the example 'find issues in projectA or projectB' and 'project in (projectA, projectB)', and 'not in' with the example 'find issues not in projectA or projectB' and 'project not in (projectA, projectB)'.

in
find issues in projectA or projectB
project in (projectA, projectB)

not in
find issues not in projectA or projectB
project not in (projectA, projectB)

Operators: is AND is not

- Used to determine if a value has been set
- The values that follow must be Empty or NULL



The image shows a screenshot of a JIRA search bar on the left and a yellow box on the right containing examples of the 'is' and 'is not' operators.

Search Bar: The search bar has a red 'x' icon and the text 'project |'. Below the search bar, a list of operators is displayed: '=', '!=', 'is not', 'is', 'not in', 'in', and 'Syntax Help'.

Yellow Box:

is
find issues that have no assignee
assignee is EMPTY
assignee = EMPTY

is not
find issues that have an assignee
assignee is not EMPTY
assignee != EMPTY

Operators: contains (~) and (!~)

- Used with text fields only (summary, description, environment, comments, custom text fields)



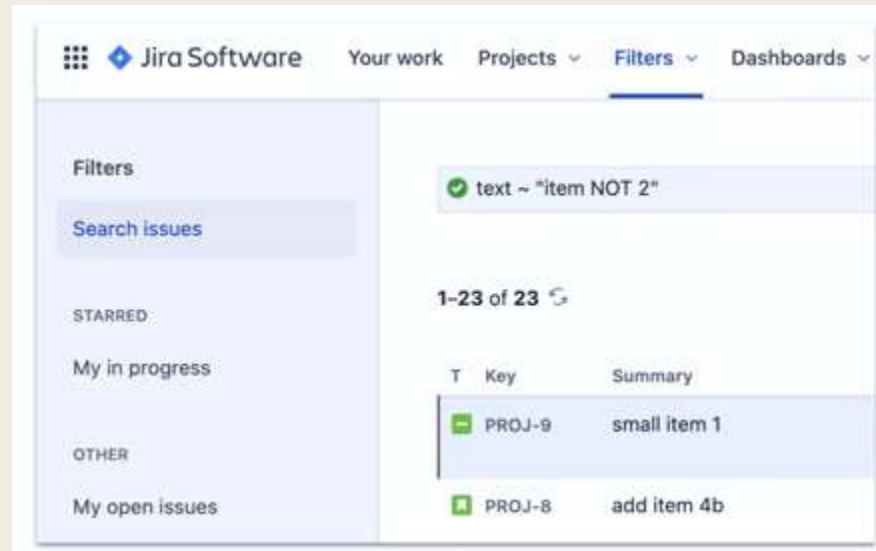
The screenshot shows a search interface with a dropdown menu for the 'summary' field. The dropdown lists the following operators: ~, !~, is not, is, and Syntax Help. To the right of the dropdown, a yellow box provides detailed information about the '~ (contains)' and '!~ (does not contain)' operators.

~ (contains)
summary contains the text "item 1"
`summary ~ "item 1"`

!~ (does not contain)
summary does not contain "item"
`summary !~ item`

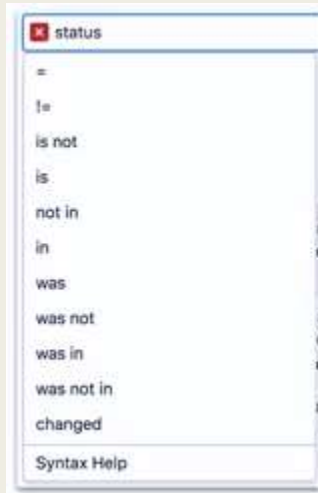
Searching Text Fields Using JQL

- The keywords in these text fields searches are an exception to the general rule that Jira searches are case insensitive.



Operators: was AND was not

- Find issues that previously had a field value



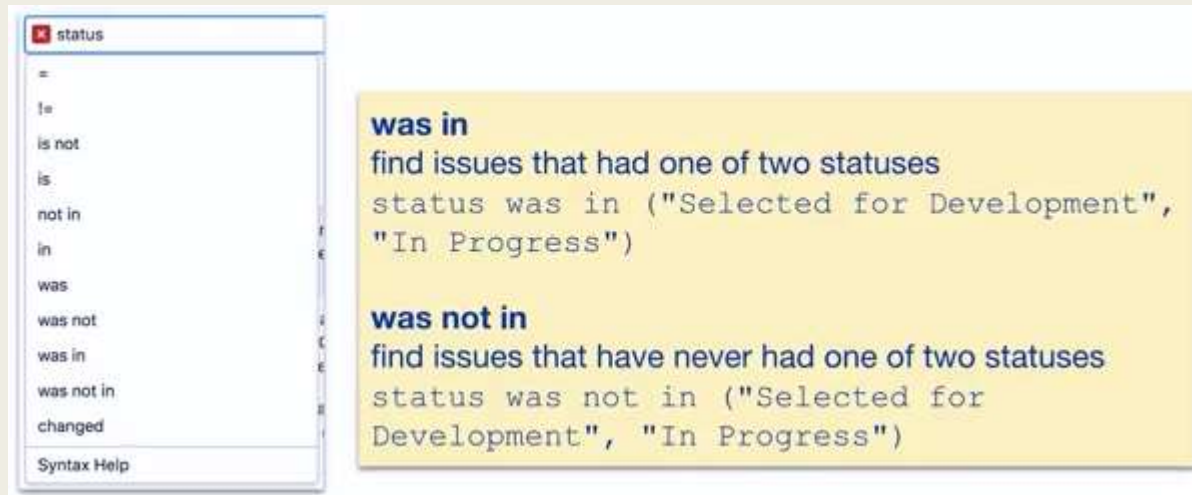
The screenshot shows a search bar with the word 'status' entered. A dropdown menu is open, displaying a list of operators: '=', '!=', 'is not', 'is', 'not in', 'in', 'was', 'was not', 'was in', 'was not in', and 'changed'. At the bottom of the dropdown is a link for 'Syntax Help'.

was
find issues where the current user has been assigned
`assignee was currentUser()`

was not
find issues that have never had a status of "In Progress"
`status was not "In Progress"`

Operators: was in AND was not in

- Issue had a field value from a set of values



The image shows a software interface with a list of operators on the left and a detailed explanation on the right. The list of operators includes: =, !=, is not, is, not in, in, was, was not, was in, was not in, and changed. The 'was in' operator is highlighted in blue. The detailed explanation on the right is in a yellow box and contains the following text:

was in
find issues that had one of two statuses
status was in ("Selected for Development",
"In Progress")

was not in
find issues that have never had one of two statuses
status was not in ("Selected for Development", "In Progress")

Predicates for “was” Related Queries

- Used to narrow results from was, was not, was in, and was not in operators

- AFTER "date"
- BEFORE "date"
- DURING ("date1", "date2")
- ON "date"
- BY "username"

find all issues moved to the Done status by the current user in the past month

```
status was Done BY currentUser() AFTER -1M
```

Operators: changed

- Issues have a field value that has changed



status

- =
- !=
- is not
- is
- not in
- in
- was
- was not
- was in
- was not in
- changed

Syntax Help

find issues whose assignee has changed
assignee changed

predictates:

- AFTER "date"
- BEFORE "date"
- DURING ("date1","date2")
- ON "date"
- BY "username"
- FROM "oldvalue"
- TO "newvalue"

status changed FROM "Done" TO "In Progress"

Boolean Operators

- AND, OR, NOT

```
assignee = currentUser() AND status = "In Progress"

status = "Selected for Development" OR status = "In Progress"
status in ("Selected for Development", "In Progress")

NOT status = Backlog
status != Backlog

find unresolved issues in all projects except SampleA
resolution = Unresolved AND NOT project = SampleA
```

Parentheses with Boolean Operators

- Use parentheses to set Boolean operator precedence and improve query readability
- Parentheses take precedence
- AND normally has precedence over OR

summary ~ "item" AND (status = "Selected for Development" OR status = "In Progress")

1-8 of 8

T	Key	Summary	Assignee	Reporter	P	Status
	PROJ-8	add item 4b	Unassigned	Steve Byrnes	↑	IN PROGRESS
	PROJ-3	add item 3	Steve Byrnes	Steve Byrnes	↑	SELECTED FOR DEV...
	PRJCT2-6	add item 6	Unassigned	Steve Byrnes	↑	SELECTED FOR DEV...

```
status = Backlog OR status = "To Do" AND summary ~ item
```

is equivalent to:

```
status = Backlog OR (status = "To Do" AND summary ~ item)
```